

Sub-Two-Bit Ternary Weight Storage with Table-Free Arithmetic Decoding for CPU Inference

Justus Theile

Independent researcher

justus.theile@gmail.com

<https://github.com/purpleskull11/bitnet-t5b>

September 2026

Abstract

Ternary (1.58-bit) large language models are stored in practice at two bits per weight, 26 % above the information content of a ternary symbol. **The code that removes that redundancy is not new, and neither is the way to decode it.** Five ternary values as the base-3 digits of one byte at 1.600 bits per weight is `llama.cpp`'s `TQ1_0`, is published with a correctness theorem as Spectra 1.1's `TQ1` [7], appears a year earlier in statistical genomics [9] and fifteen years earlier in heuristic search [8]; recovering all five digits *independently* from the packed byte, table-free, is NTRU's since 2019 [10] and is what `TQ1_0`'s own NEON path does; contracting on the digit planes without materialising a weight is `TQ1_0`'s on both instruction sets; carrying the scale per tensor rather than per block was `block_q1_3`, on the pull request that became `TQ1_0`; and the condition under which a denser format pays is Equation 1 of Zukowski et al. [11], twenty years old. §1.1 states each concession and what remains beside it.

What this paper offers instead is measurement. We build **t5b** — the same construction at a block length of $160 = 32 \times 5$, which takes no remainder byte, with the scale in `i2_s`'s existing 32-byte tail so that no graph node changes — and integrate it into `llama.cpp` as a distinct `ggml` type alongside the existing two-bit format, then measure what it is worth. The AVX2 realisation of the depth-one decode through the high half of a 16-bit multiply is the one instantiation upstream does not have, and names as an open question in its own source. We answer that question by transplanting the form into upstream's kernel and measuring it there, where it turns out *not* to pay — for a reason that is a property of upstream's encoding rather than of the idea, and that does not transfer to this one. The profitability condition is measured rather than assumed, one level down the memory hierarchy from where it was first stated, with both of its terms taken from the part the kernel runs on. So measured, **t5b** reduces the shipped **BitNet-b1.58-2B-4T** checkpoint from 1.10 GiB to 1.00 GiB and yields, under **llama-bench** *at four threads*, $1.132\times$ prompt and $1.143\times$ generation throughput, faster in five of five invocations. The re-encoding is lossless and the output is bit-identical: perplexity over 20,480 tokens of WikiText-2 agrees with the two-bit baseline to six significant figures. We further report that 4.76 % of the model's feed-forward neurons are identically zero, that the dead index sets of the gate and up projections coincide in all thirty layers, and that an independent fine-tune revives none of them. All results are from a single AVX2 microarchitecture *without* VNNI. Two independent runs bound that limitation: a static pipeline model, calibrated against the host to 3.0 %, finds the arithmetic ratio stable across Zen 3/4/5, Ice Lake and Sapphire Rapids, and a reviewer's Intel Xeon reproduces it at 2.11 against our 2.43. On real VNNI hardware the same reviewer measures the ratio rising by 8.8 % — the direction predicted, from the predicted mechanism, at a magnitude smaller than the eight-bit storage choice of a system built for those targets would suggest, though on a shared host whose spread exceeds the effect.

1 Introduction

Quantisation to ternary weights $w \in \{-1, 0, +1\}$ preserves task performance at the two-billion-parameter scale while removing multiplication from the dominant matrix operation. The resulting models are attractive for CPU deployment: the matrix–vector product requires only sign application and integer accumulation, and the weight matrix is small enough to change which hardware resource binds the computation.

That last point is usually made as a footprint argument and left there. We take it as the design constraint. During autoregressive generation every weight is read exactly once per emitted token and used exactly once, so arithmetic intensity is fixed at one multiply–accumulate per stored weight and the achievable token time is bounded below by

$$t_{\text{token}} \geq \max\left(\frac{N\mu}{\pi}, \frac{N\beta}{8B}\right), \quad (1)$$

where N is the number of weights, β the bits used to store each, μ the instructions issued on the binding execution port per multiply–accumulate, π that port’s throughput, and B the memory bandwidth available to the executing core. The first term is arithmetic; the second is transport.

Existing ternary CPU formats occupy the extremes of the (β, μ) trade-off that (1) describes. `llama.cpp`’s `i2_s` uses $\beta = 2$ with a decode of three shifts and four masks per 128 weights. `bitnet.cpp`’s TL2 and the T-MAC system reach $\beta \approx 1.667$ by table lookup, paying memory for the tables (TL1 is at 2.0); `llama.cpp`’s own TQ1_0 reaches 1.6875 by base-3 packing without a table, and TQ2_0 2.0625; Spectra 1.1 [7] publishes that family with a correctness theorem and CPU kernels as TQ1 and TQ2, recommending exactly the $p = 8, k = 5$ this work uses. Every format named so far is lossless. In the lossy direction, *Sherry* [17] reaches 1.25 bits by imposing 3:4 fine-grained sparsity on the ternary weights and packing four of them into five bits, restoring power-of-two alignment at the cost of a training-time constraint and of exactness — a different trade from the one studied here, where the weights are unchanged bit for bit. In the opposite direction, *Litespark* stores ternary weights at $\beta = 8$, on the explicit grounds that “using 2-bit packing would require unpacking weights before every computation, negating the performance benefit” — a decision made for targets whose dot-product instruction (VPDPBUSD, ARM SDOT) consumes eight-bit operands.

The base-3 packing itself is not new. `llama.cpp` has carried TQ1_0 since August 2024 (pull request #8151): its block structure in `ggml-common.h` is annotated “5 elements per byte ($3^5 = 243 < 256$)”, it decodes without a table, and it has an AVX2 dot product. TQ1_0 reaches 1.6875 bits per weight only because each 256-element block carries an f16 scale and a 16-element remainder at two bits; the code bytes themselves are at 1.600. Any claim of novelty for the packing would survive on the arithmetic of that block overhead alone, which is not a claim worth making. We had integrated into this very codebase without checking it, and record that here rather than in a footnote.

What this paper contributes is therefore narrower than any of that, and §1.1 concedes each piece to its owner. What is left: a block length of $160 = 32 \times 5$, which takes no remainder byte where TQ1_0’s 64 does; a scale carried in `i2_s`’s own 32-byte tail, so the format is drop-in for the `i2_s` path of `bitnet.cpp` with no graph node and no loader change; the AVX2 instantiation of a depth-one decode that upstream has only on NEON; a column-blocked matrix–matrix kernel, which the nearest prior format has no equivalent of; the *measurement* of a twenty-year-old condition at a level of the hierarchy where it had not been measured; and an end-to-end evaluation on a real ternary checkpoint against the format that ships with it.

What this is and is not. The mechanism is a *lossless re-encoding* of weights that are already ternary, together with a decode-fused kernel. No weight value changes; the output is bit-identical.

It is therefore neither pruning nor weight sharing nor a new quantisation scheme, and it composes with all three.

1.1 Contributions

1. A 1.600-bit variant of the *known* base-3 packing (§3). Five ternary values as the base-3 digits of a byte is not ours: it is TQ1_0’s construction in `llama.cpp`, and it is published with a correctness theorem by Vaidhya et al. [7] as Spectra 1.1’s TQ1, whose $p = 8$, $k = 5$ recommendation and 1.6-bit figure are the same as ours. Nor does it originate in this field: Breyer and Korf store heuristic estimates modulo three at 1.6 bits per entry for pattern databases in heuristic search [8], fifteen years before ternary language models, and the `5codes` format of *miraculix* publishes the same construction in statistical genomics a year before TQ1_0 [9] — its Algorithm 1 writes out the positional weights $3^0, \dots, 3^4$, its source defines `BitsPerCode 1.6`, and its stated motivation is the one taken here, to keep the multiplication on the compressed data so that nothing is decompressed. Both decode by table: *miraculix* precomputes the 3^5 possible partial dot products per tile and uses the packed byte directly as the index, so it belongs with the table-lookup systems above rather than with the decode of §4.

Nor is the scale model ours, and it is in the very pull request cited above. `block_q1_3` lived on #8151 from 2024-06-19 to 2024-07-30: five elements per byte, annotated in the same words (“*5 elements per byte* ($3^5 = 243 < 256$)”) and carrying *no* scale field at all, at 1.625 bits per weight. Its scale was a separate per-layer `ggml` tensor of extent one applied after the matmul — `Qcur = ggml_mul(ctx0, Qcur, model.layers[i1].wq_scale)` in `build_bitnet`. It was replaced by TQ1_0’s per-block `f16` deliberately, so that the stock computation graph could be reused, and the cost of that choice was recorded at the time as wasting 0.0625 bits per weight. The same construction ships today as `IQ1_BN` in `ik_llama.cpp`.

What is left is narrower again, and worth stating with its true size. Of the 20,828,160 bytes `t5b` saves against TQ1_0, `Q1_3` already delivered 78.2%; the residual is 4,546,560 bytes, 1.03% of the TQ1_0 ternary payload and 0.38% of the file. It is a block-length choice rather than a scale model: $64 = 12 \times 5 + 4$ needs a remainder byte and lands at 1.6250, whereas $160 = 32 \times 5$ needs none and lands at 1.6000. The one thing here that is not anticipated is that the scale rides in `i2_s`’s own 32-byte tail, so no graph node and no loader change is required where `Q1_3` needed both — and that is `i2_s`’s convention, not ours. As with the packing and the decode, we found this after the fact.

2. A decode of depth one rather than depth five (§4). That a multiplication-based decode avoids division and modulo is also known: Spectra 1.1 exploits $3^5 \approx 2^8$ to extract the trits *iteratively*, $b_{i+1} = (3b_i) \wedge 0xFF$ for $i = 0 \dots 4$, a strictly serial chain of five dependent multiplies. The decode used here instead takes all four quotients $\lfloor x/3^j \rfloor$ **independently, straight from the original byte**, at one `vpmulhuw` each, which is exact because a five-trit byte satisfies $x \leq 242$ and so lies inside the exactness range of 16-bit magic multipliers — collapsing the depth-five dependence to depth one.

That arithmetic is not ours either, and at exactly these parameters. The NTRU reference implementation has recovered all five trits of a packed byte since 2019 with four independent multiplies read directly from that byte — constants 171, 57, 19, 203 with shifts 9, 9, 9, 14, which we verified to be exact floor divisions by 3, 9, 27, 81 over all 256 byte values — at dependency depth one and without a lookup table [10].¹ We found this

¹`ref-common/pack3.c`, function `poly_S3_frombytes`. It does contain a serial divide-by-three chain, but only in a leftover tail of at most three coefficients, which is not compiled at all for the `hps4096821` and `hrss701` parameter sets.

only after the fact, and record it here rather than in a footnote, as with the packing above. NTRU nevertheless materialises every coefficient into a coefficient array and defers the modulo-three reduction to a separate branch-free sweep, where the kernel here feeds the digit planes straight into the contraction. **That distinction does not rescue a novelty claim either, and the counter-example is the kernel this paper positions itself against.** TQ1_0’s NEON path derives all four scalings 3, 9, 27, 81 of the loaded byte with one `vmulq_u8` each, every one of them from the *original* vector rather than from its predecessor, extracts the digits table-free by multiplying by three and keeping the two bits above eight, and passes them directly to `vdotq_s32`. That is depth one, table-free, and without materialisation, and it has shipped since pull request #8151. Its AVX2 path does the same without materialisation but at depth *two*: lacking an 8-bit multiply it chains the scalings as shift-and-add, deriving 27 from 3 and 81 from 9 — except in its four-element tail, which reaches depth one through `_mm256_mullo_epi16` against a vector of 1, 3, 9, 27.

What is therefore left is one instantiation, not a mechanism: the depth-one form in the AVX2 *main* path, reached through the high half of a 16-bit multiply. Upstream states that as an open question in the file itself — // TODO: `can _mm256_mulhi_epu16 be faster even if 16-bits?`, `arch/x86/quant.c:1406`. We verified every sentence of this paragraph against the vendored kernel at 390c307 rather than against a description of it.

We answered that question, and the answer is no — for TQ1_0. Transplanting the depth-one form into upstream’s own kernel and measuring it there gives $0.983\times$, slower in 7 of 54 interleaved rounds across six invocations, at $131 \rightarrow 137$ vector operations and $0 \rightarrow 4$ register spills. The reason does not transfer to the format described here, and the difference is the encoding. TQ1_0 stores $\lceil 256v/243 \rceil$, a fixed-point fraction, so its digits come out by multiplying by 3^j in *byte* lanes and taking the top bits — no multiply-port instruction at all, and no reason to leave byte lanes. `t5b` stores the integer $v \leq 242$, so its digits are floor divisions, AVX2 has no 8-bit multiply, and the even/odd split and fold that costs TQ1_0 those six operations is not an alternative here but the only route. The instantiation is therefore real and is not worth what a reader might assume: it is the right choice for this encoding and the wrong one for upstream’s.

And against the serial form *this* encoding admits, depth one is free rather than expensive. A chain $q_{j+1} = \text{mulhi}(q_j, 21,846)$ over the two views is two multiplies at each of four levels, the same eight the independent form issues; what changes is only the dependence. Nor does the latency it buys bind: §7 measures the loop at 67% of its multiply-port ceiling. So the honest statement is that depth one costs nothing and relieves a constraint that was not the binding one, which is a weaker claim than an earlier draft made. No head-to-head measurement against Spectra’s kernel was made: their figures are from a Mac M4 and an EPYC part, ours from one Zen 2 host.

3. A *measurement* of the profitability condition (§5), derived from (1) and verified by measuring both π and B rather than inferring either. The criterion itself is twenty years old: Equation 1 of Zukowski et al. [11] switches between the I/O-bound and CPU-bound regimes on a test that is $S \leq \Sigma$ rearranged, and applies it as an adopt-or-reject rule for a codec. What is new is the level of the hierarchy at which it is measured — there B is disk bandwidth against a RAID array and main memory is named only as future work, here it is DRAM-to-SIMD — and the fact that both of its terms are measured on the part in question rather than assumed.
4. Integration and end-to-end evaluation (§7) in `llama.cpp`, with the two-bit path retained as a control and losslessness verified by perplexity agreement over 2.05×10^4 tokens.

5. A structural observation (§8): 4.76 % of the checkpoint’s feed-forward neurons are identically zero, the dead index sets of W_{gate} and W_{up} coincide in all thirty layers, and an independent fine-tune revives none of them.

2 Background

2.1 The information bound

A ternary weight carries at most

$$H_{\text{max}} = \log_2 3 = 1.58496 \text{ bits}, \quad (2)$$

the origin of the “1.58-bit” label, attained only for a uniform distribution. The empirical marginal over all 2,084,044,800 ternary weights of the shipped checkpoint is

$$(p_{-1}, p_0, p_{+1}) = (0.288940, 0.422109, 0.288951), \quad (3)$$

giving a zeroth-order entropy

$$H = - \sum_s p_s \log_2 p_s = 1.5603 \text{ bits}. \quad (4)$$

The excess zero mass *lowers* the floor below $\log_2 3$. Storage at $\beta = 2$ exceeds (4) by 28.2 %; the code of §3 exceeds it by 2.5 %.

That residual 2.5 % is the one place an entropy coder could still buy something, and the closest work is Tan et al. [12], which pairs an entropy study of weights from 1.5 B to 405 B parameters with a tile-level Asymmetric Numeral Systems decoder fused into the GEMM pipeline of GPU inference, reporting bit rates within 0.01 to 0.1 bits of the bound. It does not settle the question here, and the reason is worth being precise about: their format space bottoms out at INT4 and FP4 and treats no ternary format, and their central observation — that nominal formats over-allocate by a factor of 2 to 10 — does not carry over to a code already within 2.5 % of its own measured entropy. The obstacle on this target is not the coder’s rate but its cost: a variable-rate decoder must still satisfy (17) against a kernel sustaining 632 million 128-weight blocks per second, and their decoder reconstructs tiles into shared memory for tensor cores rather than contracting without materialising. Whether an interleaved rANS decoder can meet that bound on a SIMD CPU is open, and this paper does not answer it.

Whose kernel the baseline is. `i2_s` is not an upstream `ggml` type. `ggml-org/llama.cpp` carries `TQ1_0` and `TQ2_0` as type numbers 34 and 35 and nothing at 36; `GGML_TYPE_I2_S = 36` is declared in the `llama.cpp` fork that `microsoft/BitNet` pins as `3rdparty/llama.cpp`, at the commit this work builds against. Every comparison in this paper is therefore against that fork’s kernel, not against `ggml`’s, and the two ternary kernels `ggml` does carry are examined separately in §7.8. The distinction matters for where a reader looks and for where a defect would be reported.

2.2 The interface a replacement must preserve

`ggml`’s ternary kernels do not compute $\sum_i w_i a_i$ directly. They compute on the shifted code $c_i = w_i + 1 \in \{0, 1, 2\}$, the unsigned operand `vpmaddubsw` requires, and the caller removes the shift:

$$\sum_{i=1}^n w_i a_i = \sum_{i=1}^n c_i a_i - \sum_{i=1}^n a_i. \quad (5)$$

The correction depends only on the activations and is therefore evaluated once per matrix–vector product rather than once per row, $O(n)$ against $O(mn)$. Any replacement must expose the same quantity $\sum_i c_i a_i$. Identity (5) requires $c_i \leq 2$; a code $c = 3$ would contribute $3a_i$ while denoting $w = 0$. We verify in §7.1 that code 3 occurs zero times across all 2,084,044,800 weights.

3 The t5b code

3.1 Definition

A block occupies 32 bytes and carries $L = 160$ ternary weights. For byte index $j \in \{0, \dots, 31\}$ and digit index $k \in \{0, \dots, 4\}$,

$$b_j = \sum_{k=0}^4 c(w_{32k+j}) 3^k, \quad c(w) = w + 1 \in \{0, 1, 2\}. \quad (6)$$

The interleave $32k + j$ is not ours at either width. `i2_s` uses it with four two-bit fields: `dequantize_row_i2_s` sends bits $6 - 2k$ of byte j to weight $32k + j$, and its AVX2 dot pairs those four shifts with activation loads at $+0, +32, +64, +96$. `TQ1_0` uses it with *five* base-3 digits, in the same file the open question of §1.1 comes from: `quantize_row_tq1_0_ref` packs `x[m + n*32]` under the comment “*5 elements per byte, along 32 bytes*”, and its AVX2 dot contracts five digit planes of one 32-byte load against activations at $+0, +32, +64, +96, +128$ — the same stride on the same instruction set. What differs is smaller than it looks and matters for §4: both upstream formats put the earliest weight in the *most* significant field where (6) puts it in the least, and `TQ1_0` stores $\lceil 256v/243 \rceil$ rather than the base-3 value v — which is why its digits come out by a different arithmetic, and why the exactness argument of §4 applies here and not there. We adopt the interleave rather than invent it, and state it because it is what the drop-in claim rests on: digit plane k addresses 32 *consecutive* activations, so the access pattern is unchanged from *both*. The code is injective since $3^5 = 243 \leq 256$, and

$$b_j \leq 2 \sum_{k=0}^4 3^k = 2 \cdot 121 = 242. \quad (7)$$

Density is

$$\beta = \frac{8 \text{ bits}}{5 \text{ weights}} = 1.600, \quad (8)$$

a factor $F = 2/1.6 = 1.25$ fewer bytes than `i2_s`.

3.2 Optimality among aligned radix-3 codes

Packing k trits into $\lceil k \log_2 3 \rceil$ bits gives $\beta(k) = \lceil k \log_2 3 \rceil / k$. Restricting to codes that fit a machine word without straddling:

k	bits	β	fits
4	7	1.750	byte, 1 bit wasted
5	8	1.600	byte, exactly
9	15	1.667	16-bit word, 1 bit wasted
10	16	1.600	16-bit word, exactly

$k = 5$ and $k = 10$ tie at 1.600 and nothing byte- or word-aligned does better; $k = 10$ is admissible because $3^{10} = 59,049 \leq 65,536$. We implement both; §4.3 shows they differ by $1.5\times$ in throughput for reasons of SIMD lane width alone, which is why the byte variant is the one deployed.

Relation to Spectra 1.1’s Theorem 1. Vaidhya et al. [7] state the *correctness* condition for this family: packing k trits into a p -bit integer is lossless if and only if $2^p > 3^k$, and they likewise recommend $p = 8, k = 5$ for an effective 1.6 bits. The table above is the *optimality* counterpart — which admissible (p, k) minimises p/k subject to p being a whole machine word

— and every entry in it satisfies their condition by construction. We claim no priority over the condition, the recommendation or the resulting bit rate: all three are theirs, and $k = 5$ is `TQ1_0`’s construction in `llama.cpp` before either. What this subsection adds is the alignment argument making $k = 10$ the only other candidate at the same density, which §4.3 then rules out on lane width.

3.3 Tail

For n not a multiple of L the final block’s unused slots take $c = 1$ ($w = 0$) and the corresponding activations are read from a zero-padded buffer, so both terms of (5) are unaffected. Storage is

$$S(n) = 32 \left\lceil \frac{n}{160} \right\rceil \text{ bytes}, \quad (9)$$

exactly 1.600 bits/weight at $K = 2,560$ and 1.6296 at $K = 6,912$; over the whole checkpoint 1.6076 including tail waste and the 32-byte per-tensor scale record `ggml` appends.

4 Kernel

4.1 Digit recovery without a chain

With $q_j = \lfloor b/3^j \rfloor$ the digits satisfy

$$d_j = q_j - 3q_{j+1}, \quad j = 0, \dots, 4, \quad q_5 = 0. \quad (10)$$

A naive implementation derives q_{j+1} from q_j , a serial dependence of length five. We obtain every q_j directly from b , which (7) makes possible.

Proposition 1 (sufficient range for exact magic division). *Let $D \geq 2$ and let M be any positive integer with $e := MD - 2^{16} \geq 1$. Then*

$$\left\lfloor \frac{xM}{2^{16}} \right\rfloor = \left\lfloor \frac{x}{D} \right\rfloor \quad \text{for every integer } 0 \leq x < \frac{2^{16} - e(D-1)}{e}. \quad (11)$$

Proof. Write $x = qD + r$ with $0 \leq r < D$. Since $MD = 2^{16} + e$,

$$xM = q(2^{16} + e) + rM = q2^{16} + (qe + rM),$$

so $\lfloor xM/2^{16} \rfloor = q + \lfloor (qe + rM)/2^{16} \rfloor$ and it suffices that $qe + rM < 2^{16}$. Using $q \leq x/D$ and $rM \leq (D-1)M = 2^{16} + e - M$,

$$qe + rM \leq \frac{xe}{D} + 2^{16} + e - M,$$

which is below 2^{16} exactly when $xe/D < M - e$. Substituting $M = (2^{16} + e)/D$ gives $xe < 2^{16} - e(D-1)$. $\square$

The bound is sufficient, not tight. Since the operative question is finite — does the identity hold for every $x \leq 242$? — we also determine the exact first-failure point by exhaustive search over the whole 16-bit domain, and it is the exhaustive figure the implementation and its tests pin:

j	$D = 3^j$	M	e	bound (11)	first failure	margin over 242
1	3	21,846	2	$x < 32,766$	$x = 32,768$	135.4×
2	9	7,282	2	$x < 32,760$	$x = 32,768$	135.4×
3	27	2,428	20	$x < 3,251$	$x = 3,293$	13.6×
4	81	811	155	$x < 343$	$x = 485$	2.00×

The tightest case first fails at $x = 485$, exactly twice the largest legal packed byte. All four quotients are therefore exact for every input the code can produce, each costs one `vpmulhuw`, and — crucially — the four are mutually independent, so the depth-five chain collapses to depth one and latency ceases to bind. What binds instead is the total vector-operation count rather than the multiply port: §7 measures the loop at 67% of its multiply-port ceiling, so that port is idle a third of the time, and §7.7 finds that removing six vector operations of which only two are multiply-port buys 1.2254, above what either the multiply-port ratio (1.18) or the total-op ratio (1.16) predicts alone. Note $M = 811$ is not the canonical $\lceil 2^{16}/81 \rceil = 810$; the implementation uses the larger constant, which has the smaller exactness range (first failure at 485 rather than 806) and is still twice what the code can produce. The proposition covers it because its hypothesis is only $MD > 2^{16}$. The suite verifies all four identities exhaustively and reports each first-failure point, so the margin is a recorded quantity rather than an assumption. Digits then follow from (10) using shifts and subtractions ($3q = (q \ll 1) + q$) on the unconstrained integer ports.

4.2 Contraction and the accumulator bound

Digit plane k is a register of 32 bytes in $\{0, 1, 2\}$, exactly the unsigned operand `vpaddubsw` expects. That range is a consequence of (7) and not a property the decode enforces: the thirteen byte values $243 \dots 255$, which no packer of this format emits, all yield $d_4 = 3$, and a block of them would bound a lane at $2 \cdot 128 \cdot 11 = 2,816$, which the twelve-block fold derived below does not survive. What the subtraction needs is weaker and does hold unconditionally — no digit subtraction borrows for any of the 256 byte values.

$$\text{acc}_{16} += \text{vpaddubsw}(d_k, a_{32k..32k+31}). \quad (12)$$

A product is at most $2 \times 128 = 256$ — the bound the header and the code use, covering $a = -128$ — and `vpaddubsw` sums adjacent pairs, so a 16-bit lane grows by at most 512 per plane and $5 \times 512 = 2560$ per block. Folding to 32-bit every Φ blocks is safe iff

$$2560 \Phi \leq 32,767 \iff \Phi \leq 12.80, \quad (13)$$

hence $\Phi = 12$. This is a *worst-case* bound. The `i2_s` reference kernel folds every 32 blocks with four planes, up to $128 \times 508 = 65,024$, and is correct only because signed activations cancel; §7.1 shows that removing the cancellation makes it disagree with exact arithmetic in 11,998 of 12,000 rows at $K = 6,912$ — statistical rather than absolute, because with 0.78% of headroom the outcome turns on each row’s own code mean.

4.3 Why bytes and not words

The ten-trits-per-`uint16` code admits a kernel that never forms a digit. Substituting (10) into $\sum_k d_k a_k$ and re-indexing,

$$\sum_{k=0}^{K-1} d_k a_k = q_0 b_0 + \sum_{j=1}^{K-1} q_j b_j, \quad b_0 = a_0, \quad b_j = a_j - 3a_{j-1}, \quad (14)$$

so only the quotient chain and a transformed activation vector are needed, the latter computed once per matrix–vector product. The identity holds modulo 2^{32} rather than term by term: intermediate products are large and cancel.

It is nonetheless slower by $1.5\times$, for reasons of lane width rather than radix. On `uint16` lanes the contraction must use `vpaddwd`, covering 16 lanes where `vpaddubsw` covers 32, and the quotient chain competes for the same port. Per 160 weights the word variant issues 19 multiply-port instructions against the byte variant’s 13, measuring 30.04 against 33.74 GMAC/s per core (§7).

4.4 Blocking for batched operation

For n_c activation columns, decoding per column costs n_c decodes. Blocking so a decoded plane is contracted against N_C columns held in registers reduces the per-block cost from $N_C(\delta + 5)$ to $\delta + 5N_C$ multiply-port instructions, $\delta = 8$. At $N_C = 8$ this is 48 instructions per 1,280 multiply-accumulates, 26.7 MACs each against i2_s's 32. Measured at $N_C \in \{2, 4, 8\}$: 50.51, 63.54, 73.26 GMAC/s. We deploy $N_C = 8$; its register pressure (181 stack references per loop) costs less than the amortisation gains.

5 When a denser code pays

Let baseline A and candidate B store the same weights at $\beta_A > \beta_B$ with $\mu_A < \mu_B$ binding-port instructions per multiply-accumulate, and let $A_A > A_B$ be the arithmetic rates the two kernels actually attain with memory taken out of the measurement (§7). Write

$$F = \frac{\beta_A}{\beta_B}, \quad S = \frac{A_A}{A_B}, \quad (15)$$

and define the *surplus* of the baseline,

$$\Sigma = \frac{\beta_A/(8B)}{1/A_A} = \frac{A_A\beta_A}{8B}. \quad (16)$$

If $\Sigma \leq 1$ the baseline is arithmetic-bound and no $S > 1$ can win. If $\Sigma > 1$ it is transport-bound and, from (1), B is faster iff

$$\boxed{S < \Sigma} \quad (17)$$

in which case it runs a factor $\min(F, \Sigma/S)$ faster.

A is the measured rate, not the port ceiling, and the difference decides cells. (1) bounds the arithmetic term by $N\mu/\pi$, and it is tempting to read $A_A = \pi/\mu_A$ straight off the port count. That is the ceiling, not the rate, and neither kernel reaches it: §7 measures the baseline at 63% of its 130.9 GMAC/s ceiling and the packed kernel at 67% of its 50.5. Substituting ceilings would give $S = \mu_B/\mu_A = 2.60$ against the 2.43 measured, and would raise every Σ below by a factor $1/0.63$ — to 2.12 at one thread and 3.39 at four — which reverses the prediction at two and at four threads, where a loss is what §7.3 measures. Both terms are therefore taken from measured kernel rates throughout, and π/μ appears only as the ceiling each kernel fails to reach. The condition is a property of a kernel pair on a machine rather than of the instruction set alone, and both of its terms are measurable in minutes.

The criterion is not new; the measurement is. Equation 1 of Zukowski et al. [11] selects between an I/O-bound and a CPU-bound regime on the test $Br/C + Br/Q \leq 1$, which is (17) rearranged, with $S = 1 + Q/C$ the slowdown decode imposes on the compute side and $\Sigma = Q/(Br)$ the delivery-to-compute ratio at the compressed footprint. That work already uses it as an adopt-or-reject rule, solving for a break-even decompression bandwidth of 883 MB s⁻¹ and rejecting the codecs below it, and its PF0R family is designed for the same reason this decode is — no branches and no dependence between the values being decompressed, so that decoding pipelines. We claim no novelty for the criterion. What differs here is the level of the hierarchy: their B is disk bandwidth against a RAID array and they name main memory only as future work, whereas both terms below are measured DRAM-to-SIMD on the part the kernel runs on. Compression-aware rooflines have since been drawn for PDE solvers [14] and, with an area axis rather than a time axis, for CPUs with in-core matrix engines [15]; the thread-dependence of the crossover — that a denser format can fail at one core and pay at six — is likewise a published result in sparse linear algebra [16].

5.1 Measuring π

Eight independent dependence chains, so latency cannot bind, with the clock derived from a dependent add chain retiring one per cycle by construction:

instruction	Gop/s	ops/cycle
<code>vpaddubsw</code>	4.09	1.05
<code>vpaddwd</code>	4.10	1.05
<code>vpmulhuw</code>	4.09	1.05
<code>vpnullw</code>	4.08	1.04
<code>vpand</code>	15.43	3.95
<code>vpaddw</code>	11.72	3.00

Measured clock 3.91 GHz. All four multiply-class instructions land on one port at 1.05 per cycle; the rest are three to four times more plentiful. This fixes $\pi = 4.09 \times 10^9 \text{ s}^{-1}$ and identifies the binding resource unambiguously.

Methodological note. An earlier version of this measurement gave all eight chains identical seeds; the compiler proved them equal and kept one, eliminating `vpand` entirely and strength-reducing `vpaddw` into a multiply. It reported 2.98×10^6 and 9.15 ops/cycle against a ceiling near 4. Only the physical impossibility of those figures exposed the defect.

5.2 Measuring B : the surplus is a function of core count

B is a core’s *share*, not the socket’s. Running the baseline kernel at 256 KiB per thread (inside L2) and 64 MiB per thread (four times one CCX’s L3), one thread pinned per physical core:

threads	arithmetic (GMAC/s)	transport (GB/s)	Σ
1	83.70	15.47	1.35
2	162.42	24.94	1.63
3	235.14	33.00	1.78
4	287.54	38.62	1.86
6	471.74	38.86	3.03

Arithmetic scales $5.6\times$ across six cores, per core falling only 6 %. Transport scales $2.5\times$ and is flat from four cores to six. The surplus therefore grows with core count for reasons independent of any kernel.

Σ_6 is not a constant, and S sits inside its range. The six-thread entry is the only cell in which the condition is satisfied, so it carries the argument, and the figure above is a single run on a quiet host. Repeating it five times at load averages of 3.8 to 7.0 gives 1.65, 1.69, 1.75, 1.92 and 2.38, with a median of 1.75. Those runs do not refute 3.03: under contention a six-thread arm does not get six cores, and the cache-resident numerator suffers more from that than the bandwidth-limited denominator, so a contended measurement is a lower bound. What they establish is that Σ_6 is a property of the machine’s occupancy as well as of the part, and that $S = 2.43$ lies *inside* the interval it spans. At 3.03 the denser format wins at six threads; at 1.75 it does not. The sign of the one positive prediction therefore depends on the host being idle, which is a weaker statement than the table alone suggests (`results/sigma_six_threads.txt`). For t5b, $S = 2.43$ measured against $F = 1.25$; by (17) the code should lose at one, two and four threads and win at six, which is what §7.3 observes, at $0.53\times$, $0.62\times$, $0.78\times$ and $1.35\times$ respectively.

6 Experimental setup

CPU	AMD Ryzen 5 3600 (Zen 2), 6 cores / 12 threads
/proc/cpuinfo	avx2, fma; no AVX-512, no VNNI
Caches	L1d 32 KiB/core, L2 512 KiB/core, L3 32 MiB in two CCX slices
Memory	62 GiB
Compiler	gcc 13.3.0, -O3 -mavx2 -mfma -march=native
Model	microsoft/bitnet-b1.58-2B-4T, 2,084,044,800 ternary weights, 210 tensors
Second model	jpacifico/Aramis-2B-BitNet-b1.58-i2s-GGUF, independent fine-tune
Harnesses	llama-bench, llama-batched-bench, llama-perplexity

The host is shared with an unrelated production workload whose load average rarely falls below 1.8; within-arm spread ranges from 2% to 146% with load, comparable to the effect under study. Every comparison is paired with the arm order alternated, and headline figures are replicated across five independent invocations of the upstream benchmark.

7 Results

7.1 Correctness

Unit level. 184,228 assertions across five suites, zero failures; of these the 177,734 covering the format and both kernel variants are in the public repository and run from a clone with a compiler alone (`make test`). The other three suites cover parts of the wider project not published here. Kernels are checked against scalar references derived independently from the specification rather than from the implemented identities, so a shared derivation error cannot pass. Coverage includes exhaustive verification of the four magic-division identities over their whole legal range, deliberate 32-bit wraparound driven 4.96 times past 2^{31} , extreme activations ± 127 , and every block and tail boundary.

Model level, argmax. Nine prompts across two models, greedy decoding, one binary, one seed: identical output by `diff`. A negative control confirms the comparison can fail.

Model level, distribution. Perplexity over 40 chunks of 512 tokens of WikiText-2, taken from `llama.cpp`'s CI copy:

$$\text{PPL}_{i2_s} = 96.8243 \pm 3.55673, \quad \text{PPL}_{t5b} = 96.8243 \pm 3.55673.$$

Not only the final estimates but all forty running estimates agree exactly, from [1] 47.2744 to [40] 96.8243; instrumentation confirms the arms differed (51,660 calls through the new path against zero). This compares a floating-point aggregate over 2.05×10^4 forward passes rather than a decoded string.

How much that assertion count is worth. An assertion count measures effort, not power, so the suite's power is measured directly by mutation: twenty-one defects are injected one at a time into the kernel and its header — each magic multiplier perturbed, the digit multiplication changed from $3y$ to $5y$, the odd-byte view shifted by seven instead of eight, the low-byte mask narrowed, two digit planes given each other's activations, the radix changed, the digit count reduced, the accumulator fold raised from 12 to 13 and to the reference kernel's 32, and six aimed at the column-blocked GEMM path specifically, which shares no code with the per-row one and carries the `pp512` result. **Eighteen are killed.**

One of the six is worth its own sentence. The proposition of §4 gives a *sufficient* exactness range for a magic multiplier, and it is conservative: replacing 811 by 812 is guaranteed only below $x = 198$ but in fact first fails at 323, above every packed byte, so it is not a defect at all. Replacing it by 813 first fails at $x = 242$ — exactly the largest legal packed byte — so it is a defect visible on one input value in 243. The suite kills it.

The remaining three survive, and checking rather than filing them is what makes the result meaningful: all three are *exactly equivalent* on every reachable input, proved exhaustively rather than argued. Replacing the magic multiplier 811 by 810 leaves $\lfloor x/81 \rfloor$ unchanged on all 256 byte values; replacing it by 812 leaves it unchanged for the same reason; and replacing the byte-wise digit subtraction by a word-wise one changes nothing on any of the 65,536 word lanes, because no low byte ever borrows — the invariant §4.3 asserts, confirmed independently of the text that asserts it. The script carries both proofs and fails if either mutant is ever *killed*, since that would mean the kernel had lost the property the proof rests on.

Code 3. Absent: 0 of 2,084,044,800 weights across all 210 tensors, so (5) holds model-wide.

A defect in one of the two baselines. With activations uniformly at ± 127 , `llama.cpp`’s `i2_s vec_dot` kernel — the reference of §7.1, not the `llamafile_sgemm` path a real run dispatches — disagrees with exact arithmetic on 11,998 of 12,000 rows at $K = 6912$, every discrepancy a multiple of 2^{16} — the overflow (13) forbids by construction in our kernel. The prediction that overflow needs $n_b = K/128 \geq 32$, i.e. $K \geq 4096$, is confirmed on both sides: **0 of 108,000** rows at $K = 2560$ wrap under the same activations.

The shipping path does not have this defect, and reading it is what established that. `tinyBLAS_I2S_AVX` folds `int16` into `int32` once per 128-weight block, inside its block loop, where the reference kernel accumulates 32 blocks first (`group32_num = nb / 32`). There is therefore no 32-block group to overflow on the path that runs, and the finding above is a statement about the reference kernel alone. An earlier version of this paragraph said “`llama.cpp`’s `i2_s AVX2` kernel” without distinguishing the two, which reads as the stronger claim.

The two exceptions are the bound speaking, not noise, and they change how the claim must be phrased. The margin is $256 \times 127 = 32,512$ against 32,767, or 0.78 %, so whether a row overflows turns on its own code mean and a row slightly below average clears the ceiling. At 12,000 rows that tail is visible, so the correct statement is statistical rather than absolute: direction and magnitude hold exactly, “must” does not.

Whether ordinary activations reach that state is not established; the mechanism is. On random `int8` the baseline is exact on all 42,000 rows tried, which is why the two kernels agree wherever it matters.

7.2 Kernel microbenchmark

One core, weights resident in L2 so transport cannot bind:

kernel	GMAC/s	instr	mul	spills	weights	ceiling	of it
<code>i2_s</code> (the fork’s)	81.93	21	4	0	128	130.9	63 %
<code>t10</code> (word)	30.04	50.25	19.25	8.25	160	34.6	87 %
<code>t5b</code> (byte)	33.74	47	13	0	160	50.5	67 %

The ceiling is $\pi \cdot (\text{weights}/\text{mul})$. `t5b` initially measured 24.36 GMAC/s — slower than `t10` despite fewer multiplies — which no end-to-end benchmark could diagnose; disassembly showed 44 stack references in a 425-instruction function. One block per iteration with each plane contracted at the moment it exists gave 47 instructions, zero spills, 33.74 GMAC/s, hence $S = 81.93/33.74 = 2.43$.

7.3 Weight traffic of one token

Replaying one token’s matrix operations over the model’s real shapes and the full weight footprint of both arms together, 940 MB, of which the 521 MB of `i2_s` codes is what a single arm streams, best of five, arms rotated:

	threads	i2_s	t10	t5b	t5b/i2_s
	1	34.57 ms	72.73	64.70	0.534
	2	20.29	36.84	33.01	0.615
	4	13.19	18.82	16.90	0.780
	6	15.15	14.49	11.18	1.354

The crossing matches (17) with $S = 2.43$ and the Σ of §5. Across all eight paired six-thread observations the median is $1.12\times$, seven of eight favouring `t5b`; best against best, noting `i2_s` does not improve past four threads, is 13.00 ms against 11.71 ms, or $1.110\times$.

The six-thread cell is above the model’s own ceiling, and the excess is not a density effect. (17) caps the gain at $\min(F, \Sigma/S)$, which at six threads is $\min(1.25, 3.03/2.43) = 1.248$, or 1.244 if F is the byte ratio this replay actually streams. The row’s $1.355\times$ stands 8.6 % above the first and 8.9 % above the second, so taking the achieved ratio does not rescue it — it lowers the ceiling. *Nothing* about weight traffic can: $\min(F, \Sigma/S) \leq F$ at every thread count, and reproducing $1.355\times$ from bytes alone would take 1.476 bits per weight, below this checkpoint’s own order-zero entropy of (4). No lossless re-encoding of these weights reaches it.

The two figures are not the same quantity. $\min(F, \Sigma/S)$ is a quotient of two *roofline* times, each arm at the larger of its arithmetic and transport bounds; the row is a quotient of two measurements. At one, two and four threads neither arm sits more than 7 % above its own floor and the modelled ratio tracks the measured one to within 9 % — at four threads 0.767 against 0.780. At six it does not: `i2_s`’s transport floor is 13.41 ms, essentially unchanged from 13.49 at four because B is flat there, and the 15.15 ms in this row stands 13.0 % above it while its `t5b` counterpart stands 3.7 % above `t5b`’s. That is the widest divergence in the eight paired observations. Taking the medians instead gives $1.244 \times 1.064/1.207 = 1.097$ against a measured median of 1.120, and best-against-best gives $1.110\times$. (17) predicts the *crossing*, and the crossing is where it says; the size of the six-thread win is one draw’s slack above a floor, not a quantity the model claims to give.

7.4 End to end in `llama.cpp`

	size	pp512 (t/s)	tg128 (t/s)
<code>i2_s</code>	1.10 GiB	119.65 ± 2.27	22.56 ± 1.26
<code>t5b</code>	1.00 GiB	135.45 ± 1.21	25.78 ± 0.87
ratio	0.909	1.132	1.143

Faster in five of five invocations on both measures; per-invocation ratios 1.096, 1.112, 1.119, 1.253, 1.132 (prompt) and 1.099, 1.133, 1.123, 1.242, 1.143 (generation).

The four-thread result contradicted §5’s prediction; the contradiction is now resolved, and the prediction was right. From $S = 2.43$ and $\Sigma_4 = 1.86$, (17) predicts a loss at four threads and the isolated replay of §7.3 delivers one ($0.780\times$); the model gains $1.132\times$. Instrumenting the glue with a cycle counter — `rdtsc` at the invariant TSC rate of 3.599978 GHz, not the core clock, with `llama-bench`’s untimed warmup removed by differencing $r = 2$ against $r = 10$ — gives the matmul time directly for one arm and by difference for the other, since everything else a token does is identical between them:

kernel	in situ	replay	ratio
t5b	16.70 ms/token	16.90 ms	0.988
i2_s	21.18 ms/token	13.19 ms	1.606

The replay predicts *our* kernel to 1.2% and mispredicts `llama.cpp`'s `i2_s` path by 61%. The effective in-situ ratio is $S_{\text{eff}} = 2.43/1.606 = 1.51$ against $\Sigma_4 = 1.86$, so $S < \Sigma$ holds at four threads and (17) predicts the observed gain rather than contradicting it.

What does not survive is the use of the reference `i2_s` kernel as a stand-in for `llama.cpp`'s real performance: it is $1.6\times$ faster than what the model runs, so every ratio computed against it — the 2.43 included — flatters the baseline. No end-to-end number changes. The correction is *the baseline's*, and the table above says so: the replay predicts t5b's matmul time to 1.2% and misses `i2_s`'s by $1.606\times$. In share terms, matmuls are 49.5% of a baseline token where the replay implied 30.8%; for t5b the replay's 44.1% was already right against 43.6% measured. Stating this as a single unlabelled figure — as an earlier draft did, taking t5b's in-situ share against `i2_s`'s replay share — compares two arms and understates how arm-specific the error is. Why the in-situ path costs $1.6\times$ its reference is not established; the dispatch, the accumulator fold and the per-column post-processing are candidates, and isolating them would mean instrumenting the control arm.

The 1.606 is weaker than the table makes it look, and the integration now carries the fix. It is not measured but derived, from two `llama-bench` runs taken separately on a host whose load ranged from 2 to 10 (4.905s against 5.479s per repetition), so every fluctuation *between* those runs lands entirely in the `i2_s` matmul figure. The integration therefore probes **both** arms at the same dispatch site in the same expression, so the ratio becomes a quotient of two measured quantities and whatever the dispatch costs is common to both and divides out.

The probe's cost is measured rather than assumed. An `rdtsc` pair alone costs 19ns and is flat in thread count. The same pair with an atomic add to *one shared counter* — what the original instrumentation used — costs 18.7ns at one thread and 87.6ns at six, a factor of 4.7, because every `ggnl` worker issues a read-modify-write to the same cache line and they serialise on the coherence protocol. Padding the counter per thread returns it to 19ns, flat. Against the 94.8µs a single matmul call takes, even the worst case is 0.09% per probe and 0.18% for the entry/exit pair, so no version endangers the measurement — but the shared counter's cost *grew along the thread axis this section compares on*, and a systematic error tracking the independent variable is worth removing at any size.

And it has now been run (`results/insitu_symmetric.txt`). The measurement needed a patched `llama.cpp`, which had meant a container this host does not grant; building it natively instead — `cmake` and `gcc`, no daemon, no privileges — removed that obstacle. `insitu_measure.sh` interleaves the two arms one repetition at a time rather than running them as two blocks, so a load excursion reaches both at comparable rates, and forms the ratio within each round.

Nine interleaved rounds at four threads, host load 3-5:

	derived (§7.4, earlier)	measured
i2_s matmul, ms/token	21.18	21.67
t5b matmul, ms/token	16.70	16.88
ratio	1.268	1.257 (median)
rounds with i2_s slower	—	9 of 9

The derived figures survive. The difference method was the right thing to object to and it gave the right answer: 2.3% on the `i2_s` time, 1.1% on t5b, 0.9% on the ratio, with the direct measurement never once favouring the other arm across nine rounds. What the objection

bought is not a corrected number but a number that no longer rests on the assumption that everything outside the matmuls costs the same in both arms.

Where that time goes. Two further probes inside `tinyBLAS_I2S_AVX`, one around the contraction loop and one around the per-column post-processing, separate what the dispatch-site probe cannot. Dispatch is the remainder, measured by difference, because a probe around the call would be the call. Five runs:

part of the <code>i2_s</code> matmul	share
contraction loop	94.4 %
post-processing	3.1 %
dispatch (by difference)	2.5 %

All three candidates are now answered. Dispatch and post-processing together are 5.6 %, so neither carries the difference: the 1.257 is a statement about the inner loop’s own arithmetic and memory traffic. The third, the accumulator fold, is answered by reading the kernel rather than by a third probe — `tinyBLAS_I2S_AVX` has *no* 32-block fold. It folds `int16` into `int32` once per 128-weight block, inside its block loop. So the fold is not a periodic cost that occasionally interrupts the contraction; it is part of every block, and it is inside the 94.4 %.

That also settles which of the two `i2_s` implementations the overflow of §7.1 belongs to, and sharpens why it happens. The kernel feeds `vpaddubsw` with raw 2-bit codes against `int8` activations, so an `int16` lane takes at most $2 \cdot 3 \cdot 127 = 762$ per instruction and $4 \cdot 762 = 3048$ per block: **ten blocks may be accumulated, not thirty-two**. The reference kernel’s 32 is three times its own bound, which is the overflow; the dispatched kernel’s 1 is a tenth of it.

The nine omitted folds cost 7.5 % of the dispatched path’s matmul time. A tree identical but for accumulating ten blocks before folding measures 9.49×10^9 cycles against 10.25×10^9 , three runs each, with *identical* perplexity — 108.6810 on four chunks of WikiText-2 in both. That is one measured component of the 25 % by which the dispatched path trails t5b, not an explanation of it, and it is not a proposal: the `i2_s` path is the control arm of every comparison here, the modified kernel lives in its own tree, and no reported ratio is built from it.

These probes are compiled out by default, and that is not tidiness. They fire once per $RM \times RN$ tile rather than once per call, and they instrument one arm only, so leaving them in inflates the `i2_s` side of the ratio — measured at 1.379 against 1.257 on the same host, a 10 % artefact of the instrument. The decomposition and the ratio come from two builds by design.

One thing this still does not settle: it is one host, the same shared machine as everything else in §7.

Prompt length. 1.106, 1.152, 1.180, 1.113 at 64, 256, 1024 and 2048 tokens: the advantage does not decay with length. We had predicted decay on the grounds that a prompt pass is arithmetic-bound; at these shapes the weights still stream from memory, so the byte saving remains live and the prediction was wrong in its premise.

Concurrency. Total throughput at 1, 2, 4, 8 parallel sequences: 1.041, 1.041, 1.027, 1.097. The advantage *grows*: N concurrent sequences turn generation into a matrix–matrix product with N columns, across which the decode amortises.

Across checkpoints. On the independent fine-tune: identical output on three prompts, 1.120× prompt and 1.075× generation.

Across depth. Repeating the block stack yields a 4.50 B-parameter, 60-layer graph `llama.cpp` loads and runs with real attention and KV cache: $1.165\times$ and $1.129\times$, both margins clear of their error bars. That artefact is *not a model* — the same thirty blocks run twice — and no quality claim is made from it.

7.5 Two ratios a reader will ask about immediately

Why 19.6 % fewer weight bytes give 14 % more throughput. Both halves of that question come from one run. The `rdtsc` counter of §7.4 times the matmuls of a `llama-bench tg128` invocation at four threads, and the same invocation’s wall clock gives the throughput, so the decomposition and the ratio it explains are not assembled from different runs or different arms. Per token:

	wall	matmuls	matmul share	the rest
i2_s	42.80 ms	21.18 ms	49.5 %	21.6 ms
t5b	38.32 ms	16.70 ms	43.6 %	21.6 ms

The last column is one quantity rather than two — attention, the KV cache, RoPE, the norms, the activation quantisation, thread barriers and graph overhead, none of which touches the weight format — and it is identical between the arms, which is what makes the decomposition testable rather than decorative. Matmuls are 49.5 % of a *baseline* token; t5b removes 21.1 % of them, which is 4.48 ms, which is 10.5 % of the baseline token, which predicts 1.117 end to end. The same run’s wall clock gives $5.479/4.905 = 1.117$. The decomposition reproduces the ratio it is supposed to explain to 0.02 %.

Two figures in that chain deserve their arm stated, because taking the wrong one is the mistake this paragraph previously made. 43.6 % is the matmul share *after* the saving, so it answers “what does t5b spend its time on” and not “why is it faster”; the second question takes 49.5 %. And an independent instrument agrees: the byte ratio $521,011,200/418,775,040 = 1.244$ against the clock’s 1.268, two measurements of the same quantity that share no apparatus, differing by 1.9 %.

The standalone replay of §7.3 puts the baseline’s matmuls at 13.19 ms against the 21.18 ms measured in situ. That gap is the replay’s, not the model’s: it drives `bitnet_vec_dot_i2_i8_s_reference`, which §7.4 measures at $1.6\times$ the speed of the path `llama.cpp` actually dispatches. The replay is a lower bound on what the matmuls cost and was read as an estimate of it. Read as bandwidth, the weight stream moves 521.0 MB in the 21.18 ms the baseline spends in the matmuls — 24.6 GBs^{-1} , against 25.1 for t5b over its 418.8 MB and 16.70 ms. Equal rates in both arms, at 64 % of the 38.62 GBs^{-1} this machine delivers at four threads: the format changes how many bytes cross, not how fast they cross, and neither arm is at the wall.

Why the file shrinks by only 9 %. The ternary tensors are 521.0 MB of a 1,187.8 MB file. One f16 tensor — the 128k-vocabulary embedding — is 656.7 MB, 55.3 % of the file, and does not change. The 102.2 MB saved is 19.6 % of what the format touches and 8.6 % of the file. The ternary share, and therefore the file saving, rises with model depth at fixed vocabulary.

7.6 A negative result on row tiling

Tiling four weight rows against one activation vector measures $1.19\times$ in cache and $0.52\times$ from memory. A tile of R rows by C columns loads each weight once and spends it on RC contractions; a matrix–vector product has $C = 1$, so tiling rows buys no reuse and merely reorders loads. Upstream draws the same line structurally: its `ggml_gemv_i2_i8_s` does not tile, while the 4×4 tile lives in `ggml_gemm_i2_i8_s`.

7.7 A cheaper decode, and the width at which it stops mattering

Because $q_3 = \lfloor x/27 \rfloor \leq 9$ for every byte, it is a nibble, and `vpshufb` can index it. The idiom — once a quantity is provably below sixteen, stop computing it and shuffle it — is standard and is not claimed here [13]; what is measured below is a smaller decode, not a new technique. It differs from the table-based ternary kernels named in §1 in the way that matters for their published cost: their shuffle is rebuilt per activation block and *replaces* the multiply-add, whereas these three tables are loop-invariant constants and the `vpmaddubsw` contraction is untouched. Three constant 16-entry tables then yield $d_4 = \lfloor q_3/3 \rfloor$, $d_3 = q_3 \bmod 3$ and $3q_3$ from q_3 alone, so the multiplier $M_4 = 811$ and both of its `vpmulhuw` disappear. The packed representation is untouched — same five digits, same 1.60755 bits/weight, same bytes — so the variant is a build option (`-DTERNARY_T5B_SHUFDIG`) and not a second format. It is bit-identical: the full suite of 97,529 assertions passes against it, and the two decoders agree on all 256 byte values including the thirteen foreign ones.

Under the build’s own flags the loop falls from 43 to 37 vector operations and from 13 to 11 multiply-port operations.² Fifteen interleaved rounds, with the untouched `i2_s` kernel measured in the same binaries as a control, give a median of $1.2254\times$ on the contraction, faster in 14 of 15 rounds,³ $p = 0.00098$ by an exact two-sided sign test; the control’s median is $1.0037\times$ with a round-to-round range of 5%, which is this measurement’s noise floor. S falls from 2.43 to 1.98.

No cell of the surplus table changes. (17) failed at one to four threads and still fails; it held at six and still holds. What changes is the margin at the top of that range: the four-thread cell moves from 31% short to 6.5% short. The improvement lands exactly where the format was losing and does nothing where it was already winning, because at six threads the packed kernel already sits at the memory knee and additional arithmetic cannot be spent.

It also does nothing for prompt processing, and the reason is structural rather than incidental. `ternary_t5b_gemm_avx2` decodes each block once and spreads it over C activation columns, so the decode is amortised over the strip width. Measured across widths at $K = 6,912$, the gain is $1.1608\times$ at $C = 1$ and $1.1363\times$ at $C = 4$, both in 9 of 9 rounds, and by $C = 8$ both arms saturate at the same 78 GMAC/s with 3 to 5 wins in 9 — a coin flip. Past that width the cost is the contraction and the activation stream, and the decode, which is all this change touches, has been amortised away. Only `tg128` can move.

Finally, the measurement exceeds what §9’s own model predicts: $43/37 = 1.1622$ on total vector operations, $13/11 = 1.1818$ on the multiply port, against 1.2254 measured. The instruction mix shows the multiply port falling from 13 to 11 while the shift-and-shuffle pair stays at exactly 9 — three shifts traded for three shuffles — so the distribution across ports flattens, and the loop sustains 2.30 vector operations per cycle against the baseline’s 2.18. Both loops spill zero times, so register pressure is not the mechanism. The flatter distribution is *consistent* with the residual 5% and is not isolated by this experiment; what the experiment does establish is that removing operations is not the only lever, and that the model of §9 is therefore incomplete.

7.8 Against TQ1_0, the nearest prior art

§1.1 records that the base-3 packing is TQ1_0’s, and that Spectra 1.1 [7] publishes the same $p = 8$, $k = 5$ construction with a correctness theorem. This subsection measures against TQ1_0, which is the form of it that ships in the codebase integrated into; no run against Spectra’s own kernels was made. The measurement is in `results/tq1_0_comparison.txt`. The comparison

²The count depends on `-march=native`, which the `Makefile` sets: without it the same source gives 42 and 36. The 0.2625 vector operations per weight quoted in §9 is the latter count.

³`shufdig_ab.sh` reproduces every figure in this subsection. It compiles both arms from the one file, asserts that the arm without the definition is byte-identical to the shipped kernel, chooses its core by sampling `/proc/stat` rather than by convention — a busy SMT sibling costs as much as a busy core — and prints the control beside the effect, so a reader can see the noise floor instead of being told it.

file is produced by re-encoding the same ternary values into **TQ1_0**’s block layout, with the per-tensor **i2_s** scale written into every block’s f16 field, so the two files carry identical weights.

	file	ternary payload	achieved bits/weight
i2_s	1,187,801,280	521,011,200	2.00000
TQ1_0	1,106,386,560	439,603,200	1.68750
t5b	1,085,565,120	418,775,040	1.60755

TQ1_0’s 1.6875 is 54 bytes per 256 weights: 48 for the five-per-byte codes, 4 for a 16-element remainder at two bits, and 2 for the f16 block scale. **t5b**’s 1.60755 is 32 bytes per 160 weights plus the tail waste at $K = 6912$ and one 32-byte record per tensor.

On size the honest margin is small. 4.7% of the ternary payload and 1.9% of the file. Four fifths of the saving this work reports against **i2_s** was already present in the vendored tree under an upstream type number. Of the remaining fifth, 78.2% was present in the same pull request as **Q1_3** before **TQ1_0** replaced it (§1.1). What is attributable here is 4,546,560 bytes — 1.03% of the **TQ1_0** ternary payload and 0.38% of the file — and it is the block length, $160 = 32 \times 5$ taking no remainder byte where $64 = 12 \times 5 + 4$ does.

On speed the margin is large and mostly not about the packing. Three rotated **llama-bench** invocations, all three files in each:

	pp512 (t/s)	tg128 (t/s)
i2_s	122.25 ± 1.52 , 77.06 ± 4.49 , 118.12 ± 3.26	23.61, 15.27, 23.56
TQ1_0	48.27 ± 0.73 , 43.95 ± 5.26 , 49.01 ± 0.29	21.68, 16.07, 22.27
t5b	130.65 ± 4.35 , 137.38 ± 0.52 , 128.17 ± 8.07	22.95, 26.43, 26.43

TQ1_0 loses prompt processing by a factor of about three, and the reason is structural rather than arithmetic: **i2_s** has a **llamafile_sgemm** case upstream, **t5b** has one because we wrote it, and **TQ1_0** has none — so it processes a prompt one activation column at a time. **A **TQ1_0** sgemm case, which nobody has written, would be expected to erase that margin.** At **tg128**, where no format has a batched path to exploit, all three land within about 10% and the ordering does not survive this host’s noise.

What survives. Against **TQ1_0** on this machine, **t5b** is worth 4.7% of the ternary bytes, plus the fact of being wired into the batched path. The claim that a sub-two-bit ternary packing is itself novel does not survive and is withdrawn in §1.1, along with the scale model, the interleave, the extraction arithmetic and the condition itself. What survives is narrower and is listed there: the block length that removes the remainder byte, the scale riding in **i2_s**’s existing 32-byte tail so that no graph node and no loader change is needed, the AVX2 instantiation of the depth-one decode, the column-blocked matrix–matrix kernel — which **TQ1_0** has no equivalent of — and the *measurement* of the condition rather than the condition.

8 Dead feed-forward neurons

2.9036% of all weight-matrix rows are identically zero — an output neuron that cannot fire for any input.

tensor	dead rows	share
W_{gate}	9,870 / 207,360	4.76 %
W_{up}	9,870 / 207,360	4.76 %
W_v	103 / 19,200	0.54 %
W_q	4 / 76,800	0.01 %
$W_{\text{down}}, W_k, W_o$	0	0.00 %

They concentrate at the front: 43.59 % of layer 1’s feed-forward neurons, 27.58 % of layer 2’s, 14.77 % of layer 3’s, near zero by layer 8. **In all thirty layers the dead index set of W_{gate} equals that of W_{up}** — not the same cardinality, the same indices. In a SwiGLU block, $h = W_{\text{down}}(\sigma(W_{\text{gate}}x) \odot W_{\text{up}}x)$, the two are combined element-wise, so a zero gate row renders the matching up row irrelevant and conversely; training removed them in pairs. W_{down} has none, the same fact along the other axis: there a dead neuron is a dead *column*.

They survive fine-tuning exactly: against the independent fine-tune, 210 of 210 tensors have identical dead index sets, all 19,847 rows, 100.00 %. Whatever removed them occurred in pre-training.

Skipping the rows in the kernel saves 2.4380 % of weight bytes *and* the same share of multiply-accumulates, and is unconditionally exact. Removing the neurons outright was also attempted, and the outcome is a negative result worth stating because three prior claims about it — two of them ours — were wrong.

It is expressible: `llama.cpp` reads `LLM_KV_FEED_FORWARD_LENGTH` through `get_key_or_arr` into a per-layer array (`llama-model.cpp:1117`), so the container and the hyper-parameters carry per-layer widths. It is *not* free of loader changes, which an earlier draft of this section asserted without checking: `src/models/bitnet.cpp` allocates its four feed-forward tensors with the global `n_ff`, so a file whose layers differ in width fails at load. Four lines change that, and with a scalar `feed_forward_length` the broadcast makes them a no-op for every existing checkpoint.

With those four lines the pruned model loads — 2.35 B parameters against 2.41 B, 1.08 GiB against 1.10, 60,948,480 ternary weights and 15,268,736 bytes removed. Every surviving weight is bit-identical and every attention tensor byte-identical; the feed-forward block’s int8 input is unchanged value for value across all 1,642,496 of them. But it is **not** exact at the token level, and the reason is the sub-layer RMS norm BitNet places between the element-wise product and W_{down} . That norm divides by the root mean over all 6,912 entries, so deleting d of them rescales the output by

$$\sqrt{\frac{K-d}{K}},$$

which in layer 1, where $d/K = 43.59\%$, is 0.751 and not a rounding matter. Pinning the divisor to the original width restores it to 2.5 ulp of float32 — not to zero. On three prompts the output is identical on one and diverges on two, late, inside repetitive passages where successive candidates are near-ties and the last bits decide. And it buys nothing measurable: `pp512` $119.24 \pm 3.11 \rightarrow 109.90 \pm 9.48$, `tg128` $23.49 \pm 0.51 \rightarrow 23.53 \pm 0.72$.

The honest summary is that 2.9 % of the ternary payload can be removed, that doing so costs four lines and changes the output, and that it makes the model no faster. We do not ship it.

9 Limitations

The port analysis is Zen 2 specific — but S is not. §5 measures one multiply-class port at 1.05 instructions per cycle against 3.0 to 4.0 for the cheap classes. A reviewer ran the same benchmark on an Intel part with AVX-512 and obtained 1.60 against 3.09: two 256-bit multiplier

ports rather than one. The ratio (17) turns on is therefore a property of the microarchitecture at a finer grain than the instruction set, and the surplus table would have to be re-measured on any other part before (17) could be applied to it.

The worry this raises is sharper than the port count alone, and it deserves its strongest form. Zen 2 cracks every 256-bit integer vector operation into two 128-bit micro-operations issued over four floating-point pipes, retiring roughly two vector operations per cycle; Zen 3 and later, and Ice Lake and later, execute 256 bits natively. The packed kernel spends $42/160 = 0.2625$ vector operations per weight against the baseline’s $17/128 = 0.1328$: it is the kernel that issues *more* of them. A machine doubling the vector-op ceiling therefore relieves the packed code preferentially, S could collapse, and the whole motivation for a roofline condition would be an artefact of one narrow part. It does not collapse: §9.1 puts S between 2.10 and 2.45 on Zen 3/4/5, Ice Lake and Sapphire Rapids against 2.50 on Zen 2, and §9.3 measures 2.11 on an Intel part.

One microarchitecture, and the decisive property is absent from it. All results are from a single Zen 2 part reporting `avx2` and `fma` and nothing else. `VPDPBUSD` collapses the contraction into one instruction while leaving the decode unchanged; by (16) this raises S and lowers Σ , moving (17) against a packed code. The *Litespark* system, targeting VNNI and ARM SDOT, stores ternary weights at eight bits for precisely this reason. §9.1 *bounds* that move at 0 to 14% from a static model; §9.2 *measures* it at 8.8% on hardware we do not own. **The central result is still ours only for AVX2 without VNNI** — every end-to-end number in §7 comes from this one host, and no VNNI machine has run the model. What §9.2 settles is the kernel-level ratio, not the deployed one.

9.1 How far a model can substitute for the hardware

The two objections above differ in one respect that matters: the first concerns code that exists and can be analysed, the second concerns code that does not.

We run `llvm-mca`, LLVM’s static pipeline simulator, over the inner loops `gcc -O3 -mavx2 -mfma` actually emits for both kernels, against the vendor-derived scheduler models for six targets. Re-deriving the loop shapes reproduces §7.1’s counts exactly for the baseline (21 instructions, 4 multiply-class, 128 weights) and to within one instruction for the packed kernel (46 against 47; the 13 multiplies and 160 weights are exact), the difference being a compiler version.

The model is calibrated before it is used. Its Zen 2 prediction is $S = 2.501$ against the $S = 81.93/33.74 = 2.428$ measured on this host: an error of 3.0%. The script exits non-zero should that drift past 8%. Absolute cycle counts are uniformly optimistic by 6 to 11% (18.01 predicted against 19.7 measured per 160-weight block), which is why only the ratio is carried forward.

target	packed, cyc/block	baseline, cyc/block	S
znver2 (this host)	18.01	5.76	2.501
znver3	11.51	3.76	2.449
znver4	11.51	3.76	2.449
znver5	11.51	3.76	2.449
icelake-server	14.02	5.34	2.098
sapphirerapids	14.52	5.01	2.317

These are modelled numbers, and the licence for printing them is that the model was tested twice. Once in-sample, against the host it was calibrated on (3.0%); once *out of sample*, against a machine it had never seen — the reviewer run of §9.3 measured $S = 2.11$ on an Intel part, where this table predicts 2.098 and 2.317 for the two Intel targets. A model that reproduces its calibration set proves nothing; one that predicts a case withheld from it has

earned a table. Every figure here is nonetheless labelled modelled, and the sections it feeds say so at each use.

Two readings would overstate this table. `znver3`, `znver4` and `znver5` return *identical* counts, so they are one prediction and not three: it holds four distinct predictions, not six. And every newer target sits at or below the Zen 2 value rather than scattered around it — the ratio improves by up to 16% and never degrades — so what is supported is one-sided, that S does not run away, and not that S is constant.

For VNNI the same apparatus is far weaker, and we set out why rather than reporting the number alone. No compiler here targets VNNI usefully, so the two VNNI loops are written by hand: in both kernels the contraction ends in a `vpmaddubsw` followed by an accumulating `vpaddw`, and `VPDPBUSD` performs exactly that pair fused, with the same unsigned-times-signed operand convention. The substitution is argued sound — the reduction width changes from `int16` pairs to `int32` quadruples, leaving the horizontal sum invariant, and it makes the accumulator bound of §4.3 unnecessary — but that is an argument, not a test. **These loops have never been executed.** Three caveats bound what the numbers mean.

1. `VPDPBUSD` accumulates in place with latency 10 to 13, so a loop carrying one accumulator per contraction is bound by that latency until unrolled. On `sapphirerapids` this makes the hand-written VNNI *baseline* slower than the AVX2 one it replaces — 7.01 cycles against 5.01 — which is a property of the accumulator count, not of VNNI. The comparison is therefore read at the resource bound, where an adequately unrolled implementation lands.
2. That metric fails the calibration the dependency-bound metric passed: on Zen 2 it gives $S = 1.96$ against the measured 2.428, an error of -19% , because the real loop is dependency-bound. Both are reported.
3. The negative control did not fire. `llvm-mca 23.1.0` assembles and times `VPDPBUSD` for `-mcpu=znver2`, a part without the instruction, and continues to under `-mattr=-avx512vnni,-avxvnni`, assigning it latency 11 and throughput 1.00. The tool cannot distinguish a legal VNNI sequence from an illegal one and so offers no check on this subsection beyond timing. The control is kept in place and reported failing.

Subject to all three, VNNI moves S in the predicted direction — against the packed code, the baseline spending 4 of 17 operations in the contraction `VPDPBUSD` collapses where the packed kernel spends 5 of 42 — and moves it by 0 to 14%: S reaches 2.08 on `znver4` and 2.51 on `icelake-server` and `sapphirerapids`. Modest, not decisive.

None of this is a measurement. It also cannot become one by recompiling: no compiler contracts `vpmaddubsw` followed by an accumulating `vpaddw` into `VPDPBUSD` as an idiom, so an AVX2 kernel built with `-march=native` on a VNNI part contains **zero** `vpdpbusd` and measures AVX2 on that part. The instruction has to be written.

`bench_vnni.c` now writes the instruction: `i2_s` and `t5b` in both forms, the VNNI pair contracting into `int32` accumulators, which for `t5b` also removes the fold of §4.3 entirely. It verifies with `objdump` at run time that its own binary contains the instruction, checks every row against exact `int64` arithmetic before printing a rate, and exits 3 rather than run on a CPU without the feature. A second build models `VPDPBUSD` in AVX2 so the algorithm can be checked where the instruction cannot execute; it passes on all 100 rows for all four kernels and deliberately prints no timings.

9.2 VNNI, measured

A reviewer ran that benchmark on an Intel Xeon reporting `avx512_vnni`: eight invocations, the ISA check passing on each, all four kernels exact against `int64` arithmetic on every row. **This**

is the paper’s largest limitation moving from modelled to measured, and it moves only part of the way, so both parts are stated.

	AVX2	VNNI	
median S (i2_s/t5b)	2.04	2.22	+8.8 %
i2_s gain from VNNI	—	—	1.24×
t5b gain from VNNI	—	—	1.15×
runs in which S rose	—	6 of 8	

The direction is the predicted one and the mechanism is the predicted one. §9.1 argued that VPDPBUSD must help the baseline more than the packed format, because the baseline spends 4 of its 17 vector operations in the contraction the instruction collapses while the packed kernel spends 5 of 42. The measurement agrees: 1.24× against 1.15×, and S rises accordingly.

The size is modest and the spread is not smaller than the effect. S rose in six runs of eight, a majority and not a clean separation; the host is a shared two-vCPU sandbox, and the reviewer’s own caveat — that run-to-run variation there exceeds the effect being measured — is adopted rather than argued away. An 8.8 % shift in S against Σ values ranging from 1.35 to 3.08 does not by itself decide (17) either way at any thread count.

What the static model got right, and where it was pessimistic. §9.1 bounded VNNI’s cost at 0 to 14 % and this measurement lands at 8.8 %, inside that band. The model predicted $S(\text{vnni}) = 2.08$ on `znver4` and 2.51 on the two Intel targets; the measured 2.22 sits between them. Given that the model’s negative control failed and its VNNI loops had never been executed, agreement to this degree is more than was claimed for it.

What remains open is a clean machine — or a better design, which is what was done instead. Six of eight is a majority because the eight runs timed the four kernels in sequence, so the AVX2 and VNNI arms of each ratio came from different moments on a drifting host. `bench_vnni.c` now interleaves them: 31 rounds, all four kernels measured in rapid alternation within each round with the starting kernel rotated, S formed inside the round, and the result reported as a median difference with an exact two-sided sign test. A load excursion then reaches both arms of every ratio instead of landing on one block. That converts a between-condition comparison on a noisy host into a paired one, which is the design a shared machine can actually answer; the numbers above predate it and a dedicated part is no longer the only way to sharpen them. The claim this subsection supports is narrow: on real VNNI hardware the packed format’s arithmetic disadvantage grows by roughly a tenth, in the direction and for the reason §9.1 predicted, which is a smaller penalty than the eight-bit storage choice of a system built for those targets would suggest.

9.3 A second microarchitecture, measured

A reviewer ran the benchmarks on an Intel Xeon (AVX-512 capable, 2 vCPU cloud sandbox), giving a genuine second data point for everything except VNNI. Their caveats are adopted rather than paraphrased: with two vCPUs and no pinning, only the one- and two-thread rows carry information; the four-thread `bench_token` row and every `bench_threads` row from $T = 3$ are oversubscribed and mean nothing.

	this host (Zen 2)	reviewer (Intel Xeon)
multiply class, ops/cycle	1.05	1.32 to 1.44
cheap class, ops/cycle	3.0 to 4.0	2.97 to 3.04
i2_s reference, GMAC/s/core	81.93	49.86
t5b, GMAC/s/core	33.74	23.61
S	2.43	2.11
Σ at 1 thread	1.35	1.65
Σ at 2 threads	1.63	2.23

Two readings, the second load-bearing. First, S does not collapse on a wider machine: 2.11 against 2.43, two runs within 1% of each other. Second, **this is an out-of-sample test of §9.1’s model, and it passes.** That model predicted $S = 2.098$ for `icelake-server` and 2.317 for `sapphirerapids` — bracketing the measured 2.11 — from a calibration performed only against this Zen 2 host. A static pipeline model tuned on one microarchitecture predicted a second it had never seen, to within 1%.

At two threads the reviewer’s machine gives $S = 2.11 < \Sigma = 2.23$, so (17) predicts a marginal win there and none at one thread. That is a prediction and not a result: no end-to-end run was made on that host, and the elevated Σ comes from the VM’s low per-core DRAM bandwidth (8.1 GB s^{-1} single-threaded) rather than from the microarchitecture.

One model size and one architecture family. No public ternary checkpoint larger than 2B exists. The synthetic 60-layer graph doubles weight traffic but not head count, hidden size or vocabulary.

Measurement environment. The host is shared and run-to-run spread is comparable to the effect: across eight invocations at ordinary load, one has the denser code losing. The headline is therefore the five-invocation replication rather than any single run.

Verification scope. The round trip is exact on all 210 tensors and all 2,084,044,800 weights, and so — since this revision — is the dot-product comparison: 126,000 row-and-activation cases across every `i2_s` tensor, on which the packed kernel matches exact `int64` arithmetic without exception. What remains limited is the read-back from disk, covering the 210 plus the embedding rather than all 332 tensors; the converter’s own reread covers the dims, type, offset and alignment of all 332.

Two things this does *not* establish, and they are the ones to hold against it. Nothing here runs the converted model: that the weights survive repacking is not evidence that a loader reading them emits the same tokens, and it cannot be until a loader knows type 43. And exhaustiveness over tensors is not exhaustiveness over inputs — 200 rows per tensor under three activation patterns is a large sample of a space that is not finite.

Task benchmarks. None. The format question does not admit one — output is bit-identical — and where this *model* stands against others is a different question this paper does not address. We ran a cross-model perplexity comparison and do not report it as a result, for two reasons, both of which leave the identity result untouched because both arms are affected equally.

The first is metadata: the checkpoint’s GGUF requires `tokenizer.ggml.pre` and the EOS id to be supplied through `--override-kv` because its own metadata is wrong, so an absolute perplexity from it is at least as likely to reflect a tokenisation artefact as the model.

The second we found while auditing this work, and it is not a suspicion but a measurement: the graph `llama.cpp` builds for this architecture is not the model’s. `build_ffn` is called with `LLM_FFN_SILU` in `src/models/bitnet.cpp`, whereas the checkpoint’s own `config.json` declares

"hidden_act": "relu2" and its technical report [2] states that “instead of the commonly used SwiGLU activation ... BitNet b1.58 2B4T employs squared ReLU”. The GGUF does not settle it: it declares `general.architecture = bitnet-b1.58` and carries no activation key at all, so the per-architecture default in the loader is the only thing choosing, and `LLM_FFN_RELU_SQR` exists beside it in `src/llama-graph.h` and is used by five other architectures.

Substituting it is one token of diff, and we ran it. Over the same forty chunks of WikiText-2:

	i2_s	t5b
LLM_FFN_SILU (as built)	96.8243 $\pm$ 3.55673	96.8243 $\pm$ 3.55673
LLM_FFN_RELU_SQR	16.7482 $\pm$ 0.50521	16.7482 $\pm$ 0.50521

A factor of 5.78. The 96.8243 quoted above is therefore a property of a mis-built graph and not of this checkpoint, which is the second and larger of the two reasons it is not reported as a result.

The identity survives the change, and that was the check that could have failed. The two arms agree to six significant figures under *both* activations — they have always run the same graph, so a wrong activation cancels exactly, and a divergence here would have meant something other than the activation was wrong. Nothing else in this paper is affected: every rate, every density figure and the bit-identity result are comparisons between two formats under one loader.

Engineering. The blocked matrix–matrix kernel spills 181 times per loop at its chosen width; narrower widths spill less and measure slower. The type identifier used (43) is not reserved by upstream, so a stock `llama.cpp` cannot read the file by construction rather than by accident.

10 Conclusion

The gap between the 1.58 bits a ternary weight carries and the 2.00 it is stored in is recoverable on commodity AVX2 hardware without lookup tables. Five ternary values fit a byte at exactly 1.600 bits; all four radix-3 quotients are extractable from that byte with one independent multiply each, because the byte is bounded by 242 and lies deep inside the exactness range of 16-bit magic multipliers; and the contraction proceeds on the digits, so no weight is materialised.

Whether the resulting 25 % reduction in weight traffic is profitable is not a property of the code. It is decided by (17), a comparison between a core’s multiply-port throughput and its share of memory bandwidth, both measurable in minutes. On the part measured here the condition holds from four cores upward, and the deployed model is 9 % smaller and 13 % to 14 % faster with bit-identical output.

The same inequality predicts the result will narrow on a part with VNNI, and §9.2 measures that narrowing at 8.8 % — the direction and mechanism the condition names, at a magnitude the eight-bit storage choice of a system built for such parts would not have led one to expect. That is not a caveat appended to a positive result; it is the result, which is that weight density is a hardware-dependent optimisation with a computable decision rule rather than a property of the model. What §9.1 does settle is that the rule’s input is not parochial: across four distinct scheduler models spanning two vendors and three generations, S ranges from 2.10 to 2.50 and every newer target sits at or *below* the Zen 2 value it was derived on, by at most 16 %. The ratio never worsens on a wider machine, so the rule is worth applying rather than an accident of the one part that produced it.

Corrections

Fifteen claims in earlier drafts of this paper were wrong and were corrected before submission — among them the novelty of the packing, of the depth-one extraction arithmetic, of the fused digit-plane contraction and of the per-tensor scale model, two statements about dead-neuron

removal, and the assertion that a run on VNNI hardware would settle §9.1. Each is listed with what replaced it and how it was found in `CHANGELOG.md` in the repository below, kept there rather than here so that this document states what holds.

What a $\pm$ means here. Three different things, and the document should say which. Around an activation bound it is the int8 range. After a perplexity it is `llama-perplexity`’s own figure, the standard error of the mean over chunks: it computes $\sqrt{(\overline{v^2} - \bar{v}^2)/(n - 1)}$ on the per-chunk negative log-likelihood and scales it by the perplexity. After a throughput it is `llama-bench`’s own figure over its repetitions. None of them is a confidence interval and none is comparable to another; each is reproduced verbatim from the tool that printed it, and every pair in this paper was checked against the evidence file it came from.

Data and code availability

The format, the kernels, the benchmarks, the converters and every evidence file cited here are public under the Apache License 2.0 at

<https://github.com/purpleskull11/bitnet-t5b>

with the layout below. No model weights are distributed and no upstream source is redistributed verbatim; the integration is applied as a patch to a checkout the user supplies, and the files that reproduce or adapt an upstream contract say so in their own headers and in `NOTICE`.

claim	file
code histogram, code 3 absence, entropy	<code>results/tensor_structure.txt</code> , <code>results/real_weights_check.txt</code>
instruction-port throughput π	<code>benchmarks/bench_ports.c</code>
surplus Σ against core count	<code>results/thread_headroom.txt</code> , <code>benchmarks/bench_threads.c</code>
kernel rates, instruction and spill counts	<code>results/weight_density.txt</code> , <code>benchmarks/bench_alu.c</code>
weight traffic of one token	<code>benchmarks/bench_token.c</code>
end-to-end throughput	<code>results/inference_t5b.txt</code>
in-situ matmul time, both arms	<code>results/insitu_symmetric.txt</code> , <code>results/insitu_kernel_rate.txt</code>
matmul share of a token	<code>results/insitu_kernel_rate.txt</code>
against TQ1_0 at equal weights	<code>results/tq1_0_comparison.txt</code> , <code>tools/gguf_to_tq1_0.py</code>
perplexity agreement between the formats	<code>results/perplexity_t5b.txt</code>
round trip on real weights	<code>results/real_weights_check.txt</code>
dead feed-forward neurons	<code>results/dead_neurons.txt</code> , <code>tools/dead_neurons.py</code>

The format and its kernels are `src/ternary_t5b.{c,h}` with the `ggml`-facing contract in `src/ggml_t5b_glue.{c,h}` and the suite in `src/test_t5b.c`; the word-lane variant of §4.3 is `src/ternary_t10.{c,h}`; the converter is `tools/gguf_to_t5b.py`.

What the public repository does not contain. The measurements that need a model — §7’s end-to-end table, the perplexity agreement — additionally require a `llama.cpp` build carrying the type, a converted GGUF and the checkpoint itself. The integration is supplied as a patch against a named upstream commit rather than as copied sources, and the `i2_s` reference kernel used as the baseline is fetched from that commit by a script rather than redistributed. Everything else — the packing, the kernels, the suite, all four benchmarks, the converters and every evidence file — builds and runs from a clone with a compiler and Python alone.

References

- [1] S. Ma, H. Wang, L. Ma, L. Wang, W. Wu, P. Dong, L. Zhang, J. Xue, F. Wei. *The Era of 1-bit LLMs: All Large Language Models are in 1.58 Bits*. arXiv:2402.17764, 2024.
- [2] Microsoft. *BitNet b1.58 2B4T Technical Report*. arXiv:2504.12285, 2025.
- [3] J. Wang, H. Zhou, T. Song, S. Cao, Y. Xia, T. Cao, J. Wei, S. Ma, H. Wang, F. Wei. *Bitnet.cpp: Efficient Edge Inference for Ternary LLMs*. arXiv:2502.11880, 2025.
- [4] J. Wang, H. Zhou, T. Song, S. Mao, S. Ma, H. Wang, Y. Xia, F. Wei. *1-bit AI Infra: Part 1.1, Fast and Lossless BitNet b1.58 Inference on CPUs*. arXiv:2410.16144, 2024.
- [5] J. Wei, S. Cao, T. Cao, L. Ma, L. Wang, Y. Zhang, M. Yang. *T-MAC: CPU Renaissance via Table Lookup for Low-Bit LLM Deployment on Edge*. EuroSys, 2025. arXiv:2407.00088.
- [6] compilade. *ggml-quants: ternary packing for TriLMs and BitNet b1.58*. llama.cpp pull request #8151, August 2024.
- [7] T. Vaidhya, A. Kaushal, V. Jain, F. Couture-Harpin, P. Shishodia, M. Behbahani, Y. Nevmyvaka, I. Rish. *Spectra 1.1: Scaling Laws and Efficient Inference for Ternary Language Models*. arXiv:2506.23025, 2025.
- [8] T. M. Breyer, R. E. Korf. *1.6-Bit Pattern Databases*. Proceedings of the AAAI Conference on Artificial Intelligence 24(1):39–44, 2010. doi:10.1609/aaai.v24i1.7558.
- [9] A. Freudenberg, J. Vandenplas, M. Schlather, T. Pook, R. Evans, J. ten Napel. *Accelerated matrix-vector multiplications for matrices involving genotype covariates with applications in genomic prediction*. Frontiers in Genetics 14:1220408, 2023. doi:10.3389/fgene.2023.1220408.
- [10] J. M. Schanck et al. *NTRU reference implementation, ref-common/pack3.c*. <https://github.com/jschanck/ntru>, 2019. NIST post-quantum cryptography finalist; vendored through PQClean into liboqs.
- [11] M. Zukowski, S. Heman, N. Nes, P. Boncz. *Super-Scalar RAM-CPU Cache Compression*. Proceedings of the 22nd International Conference on Data Engineering (ICDE), 2006. doi:10.1109/ICDE.2006.150.
- [12] Y. Tan, X. Chen, G. Alonso, K.-S. Wong, B. He. *Approaching Shannon Bound with Lossless LLM Weight Compression*. arXiv:2606.15789, 2026. To appear, ISCA 2026.
- [13] W. Muła, D. Lemire. *Faster Base64 Encoding and Decoding using AVX2 Instructions*. arXiv:1704.00605, 2017. See also W. Muła, *Implementing byte-wise lookup table with PSHUFQ*, 2016.
- [14] S. Götschel, M. Weiser. *Compression Challenges in Large Scale Partial Differential Equation Solvers*. Algorithms 12(9):197, 2019. Preprint arXiv:1907.00667.
- [15] G. Gerogiannis, S. Eyerman, E. Georganas, W. Heirman, J. Torrellas. *DECA: A Near-Core LLM Decompression Accelerator Grounded on a 3D Roofline Model*. MICRO-58, 2025. doi:10.1145/3725843.3756073. Preprint arXiv:2505.19349.
- [16] J. I. Aliaga, H. Anzt, T. Grützmacher, E. S. Quintana-Ortí, A. E. Tomás. *Compression and load balancing for efficient sparse matrix-vector product on multicore processors and graphics processing units*. Concurrency and Computation: Practice and Experience, 2021. doi:10.1002/cpe.6515.

- [17] H. Huang, D. Wu, Q. Hu, G. Yu, J. Yang, J. Zhu, X. Liu, D. Wu. *Sherry: Hardware-Efficient 1.25-Bit Ternary Quantization via Fine-grained Sparsification*. arXiv:2601.07892, 2026.
- [18] *Litespark Inference on Consumer CPUs: Custom SIMD Kernels for Ternary Neural Networks*. arXiv:2605.06485.
- [19] S. Han, H. Mao, W. J. Dally. *Deep Compression: Compressing Deep Neural Networks with Pruning, Trained Quantization and Huffman Coding*. ICLR, 2016. arXiv:1510.00149.
- [20] T. Zhang et al. *70% Size, 100% Accuracy: Lossless LLM Compression for Efficient GPU Inference via Dynamic-Length Float*. arXiv:2504.11651, 2025.
- [21] G. Gerganov et al. *llama.cpp / ggml*.
- [22] S. Merity, C. Xiong, J. Bradbury, R. Socher. *Pointer Sentinel Mixture Models*. arXiv:1609.07843, 2016.